

Overture 이론적 기반 → 제품 적용 매핑

YC(이예찬)의 학술적 배경에서 도출된 이론·방법론을 Overture 제품에 적용하는 종합 문서.

작성일: 2026-03-28

출처: 서울대 '컴퓨터와 마음' 중간/기말고사 답안 (2014), 경제성장론 연구 기말보고서 (2014-15), 대화 분석

전체 흐름: 이론 → 제품 파이프라인

WHY – Overture가 존재하는 이유

- ① 상호 지탱에 의한 경직
(베이트슨 학습이론 + 퍼스 가추법 + 사이버네틱 피드백 루프)
→ 사용자-AI 간 **sycophancy**의 구조적 원인
- ② 경계가 아니라 관계를 설계하라
(베이트슨 이중기술 + ANT 번역 개념)
→ 인간-AI 협업의 설계 철학

WHAT – Overture가 측정해야 하는 것

- ③ 차이를 만드는 차이 (베이트슨 정보 이론)
→ eval이 "버튼 클릭"이 아니라 "사고 변화"를 측정
- ④ 시간의 추가분 γ (경제성장론 보고서)
→ 프로세스를 거치면서 실제로 새로운 것이 생겼는가
- ⑤ 네트워크 링크 수 (경제성장론 보고서 + ANT)
→ 하나의 통찰이 몇 단계에 걸쳐 영향을 미쳤는가

HOW – Overture가 작동하는 방식

- ⑥ 번역(translation)으로서의 페르소나 (ANT)
→ 페르소나가 승인 조건을 사용자의 계획 언어로 번역
- ⑦ 유인 양립적 자기 평가 설계 (메커니즘 디자인)
→ 사용자가 정직하게 가정을 평가하는 것이 지배 전략
- ⑧ 메타 패턴에서 마음을 찾기 (베이트슨)
→ **Concertmaster**의 교차 의사결정 패턴 감지
- ⑨ 진화적 질서에 따른 접근 (중간고사 2번 답안)
→ 판단 능력을 물질→생명→지각→인간지각 순으로 구축

META - Overture 자체의 자기 감시

⑩ 자기참조적 경직 감시

→ 시스템 자체가 sycophantic해지지 않는가

① 상호 지탱에 의한 경직

출처

컴퓨터와 마음 기말고사 답안 3번 (자아와 세계), 2014

원문

"자아는 세계를 통해 가추의 대전제들을 착안해내고, 세계는 그 대상이 됨으로서 서로를 지탱한다. 서로를 지탱함을 통해서 얻은 안정감과 체계화 때문에 이들은 경직되어 간다."

이론적 재료

- 베이트슨의 **Learning II (deutero-learning)**: 학습의 맥락 자체가 고정되는 현상. 한번 특정 방식으로 세계를 해석하는 법을 배우면, 그 해석 프레임 자체는 의문시되지 않고 굳어진다.
- 퍼스(Peirce)의 가추법(abduction): 결론에서 거꾸로 그럴듯한 대전제를 착안하는 추론. YC가 이것을 인식론에서 관계론으로 전환시킴 — 가추가 자아와 세계가 서로를 구성하는 메커니즘이 됨.
- 사이버네틱 양성 피드백 루프: 상호작용 패턴이 자기강화적으로 작동하면서 시스템이 경직되는 현상.

Overture 적용

현재 상태: Anti-sycophancy가 "반대 의견을 추가한다"는 수준에서 작동. 페르소나 반론 + Devil's Advocate.

개선 방향: 문제는 반대 의견의 부재가 아니라, 자아(사용자)와 세계(AI)가 서로를 지탱하면서 가추의 대전제 자체가 경직되는 구조. 사용자가 페르소나 피드백에 익숙해지면, 어떤 종류의 반론이 올지 예측하고, 그 예측에 맞게 계획을 조정하고, 시스템이 그걸 "수렴"이라고 판정한다. 새로운 형태의 상호 지탱.

구체적 구현:

1. 경직 감지 메커니즘 — Concertmaster에서 다음 패턴을 감지:
 - 수렴이 1회만에 완료 (너무 빠른 수렴 = 경직의 신호)
 - 사용자의 초기 프레임이 리허설 후에도 본질적으로 변하지 않음
 - 페르소나 피드백의 종류가 사용자의 예측 범위 안에 머무름
2. 경직 깨기 — 감지되면:
 - 페르소나의 관점 축을 교란 (이전에 쓰지 않았던 dimension에서 공격)
 - "이번 수렴이 이전 5회와 같은 패턴입니다. 의도적인가요?" 플래그

- Reframe 단계에서 근본적으로 다른 전략(Challenge Existence) 강제 제안

코드 앵커: `concertmaster.ts` → 경직 감지 함수 추가, `eval-engine.ts` → `convergence_too_fast`
eval 추가

② 경계가 아니라 관계를 설계하라

출처

컴퓨터와 마음 기말고사 답안 3번 마지막 문단, 2014

원문

"특히 기계 시스템과 인간 시스템의 결합이 시도되고 있는 시점에서, 인간의 마음에 대한 고정관념을 가지고 양자를 비교, 대조하여 경계를 규정하는데 초점을 맞추는 고전적 사고에서 벗어나 그 '관계'를 이해하는데 노력을 기울여야 새로운 시스템의 Mind를 이해할 수 있으리라고 예상한다."

이론적 재료

- 베이트슨의 이중기술(double description): 단안으로는 볼 수 없는 깊이가 양안의 차이에서 발생. 관계는 항상 이중기술의 산물.
- ANT(행위자 네트워크 이론): 라투르 — 행위자의 본질이 아니라 행위자 간 연결(association)이 세계를 구성한다.

Overture 적용

현재 상태: 편곡(Recast)에서 "AI가 할 것 / 사람이 할 것"을 나눈다. 경계를 긋는 방식.

개선 방향: 경계를 긋는 것에서 관계를 설계하는 것으로. "이 단계는 AI가 한다"가 아니라, "이 단계에서 AI와 사람의 관계는 이렇게 작동한다"로.

구체적 구현:

1. Actor 필드 확장 — `ai | human | both`에서 관계 유형을 추가:
 - `ai_generates_human_validates` (AI 초안 → 사람 검증)
 - `human_frames_ai_executes` (사람이 프레임 설정 → AI 실행)
 - `parallel_then_synthesize` (각각 독립 작업 → 합성)
 - `iterative_handoff` (번갈아가며 발전)
2. 관계 품질 eval — 단순히 "체크포인트를 통과했는가"가 아니라, "AI-사람 간 핸드오프에서 정보 손실이 있었는가"
3. 이중기술 적용 — 같은 과제를 AI 관점과 사람 관점에서 각각 기술하고, 그 차이에서 나오는 제3의 정보를 포착 (양안 시각의 원근감처럼)

코드 앵커: `stores/types.ts` → `RecastStep.actor_relationship` 필드 추가, `context-chain.ts`
→ 핸드오프 정보 손실 추적

③ 차이를 만드는 차이 (difference that makes a difference)

출처

컴퓨터와 마음 중간고사 답안 5-1번 (정보, 차이, 관찰자), 2014

원문 (답안의 정보 정의 종합)

정보가 산출되려면 최소 2개의 어떤 것이 동일한 dimension에서 관계를 맺고 있어야 하며, 그 차이가 관찰자(뇌, 컴퓨터) 내부에서 차이라고 확인될 때 비로소 정보가 된다.

이론적 재료

- 베이트슨의 정보 이론: "Information is a difference that makes a difference." 정보는 객관적으로 존재하는 것이 아니라, 관찰자가 차이를 차이로 감지했을 때 발생.
- 역치(threshold): 차이의 지각은 한계점에 의해 한정. 너무 작거나 너무 느린 차이는 지각 대상이 되지 못함.

Overture 적용

현재 상태: eval이 행위 발생 여부를 측정. `question_accepted` = 사용자가 버튼을 눌렀는가. 이걸 차이가 발생했는지를 측정하지 않음.

개선 방향: eval이 "차이를 만드는 차이"를 측정해야 함. 리프레임이 사용자의 mental model에 실제로 차이를 만들었는가?

구체적 구현:

1. 구조적 변화량 추적 — `reframe → recast` 사이에서:
 - 리프레임된 질문이 편곡의 governing idea를 얼마나 바꿨는가 (텍스트 유사도 역수)
 - 편곡의 step 구조가 리프레임 전후로 얼마나 달라졌는가
 - → 변화량이 0에 가까우면: 리프레임이 "차이를 만들지 못한" 것
2. 역치 적용 — 너무 작은 변화(단어 수준 수정)는 noise로 분류, 구조적 변화(step 추가/삭제, actor 변경, 가정 추가)만 signal로 포착
3. 관찰자 내부화 확인 — 사용자가 리프레임을 "수락"한 것과, 리프레임이 사용자의 이후 행동을 실제로 바꾼 것은 다름. 후자를 측정.

새로운 eval:

difference_that_makes_difference:

question: "리프레임이 이후 단계의 구조를 실제로 변화시켰는가?"

measure: recast.governing_idea의 decompose.original_question 대비 의미적 거리 > threshold

AND recast.steps 구조가 리프레임 없이 예측되는 구조와 다른가

코드 앵커: `eval-engine.ts` → `difference_impact` eval 추가, `context-chain.ts` → 단계 간 구조적 변화량 계산 함수

④ 시간의 추가분 γ

출처

경제성장론 연구 기말보고서 3장 ('시간의 증가분'), 2014-15

원문

"전체 투자 시간 대비, 새로운 결과물을 도출하는데 사용한 시간의 추가분으로 집계된다... 비록 실험이 나 연구에는 실패하였더라도, 존재하던 여러 가설을 기각해냈다면 이는 새로운 결과물로 포함한다."

이론적 재료

- 프라스카티 매뉴얼의 **R&D** 정의: 이전에 생산/연구된 것의 반복은 비R&D, 그 이후의 '추가분'만 R&D.
- 내생적 성장이론 (**Romer**): R&D 투입의 양이 아니라 질(생산성에 기여하는 정도)이 성장을 결정.
- **YC**의 독자적 제안: 머릿수(연구자 수)나 금액(투자액)보다 '시간의 추가분'이 실제 R&D 성과를 더 정확하게 반영.

Overture 적용

현재 상태: eval이 프로세스 통과 여부를 측정. "편곡으로 진행했는가?" = 머릿수를 세는 것과 같음.

개선 방향: Overture 프로세스를 거치면서 실제로 "추가분"이 발생했는가를 측정.

구체적 구현:

1. γ (Overture) 정의:

γ = 최종 산출물에서, Overture 프로세스를 거치지 않았으면 존재하지 않았을 요소의 비율

2. 단계별 γ 측정:

- 악보 해석 γ : 발견된 숨은 전제 중, 이후 단계에서 실제로 다뤄진 비율
- 편곡 γ : 설계된 step 중, 리프레임에서 나온 인사이트 때문에 존재하는 step의 비율

- 리허설 γ : 페르소나가 제기한 리스크 중, 합주 연습에서 계획을 실제로 수정하게 만든 비율
- 합주 연습 γ : 수정된 항목 중, 새로운 구조적 변화(가설 기각 포함)를 만든 비율

3. 가설 기각도 γ 에 포함: 사용자가 리허설 후 원래 계획을 폐기하고 악보 해석으로 돌아가는 것 — 현재 eval로는 "실패"이지만, γ 프레임으로는 가설을 기각한 것이므로 높은 가치의 추가분.

4. γ 기반 DQ 보정: 높은 DQ 점수이지만 γ 가 0에 가까우면 → "프로세스는 거쳤지만 사고는 변하지 않았다" 경고

코드 앵커: `decision-quality.ts` → `calculateGamma()` 함수, `eval-engine.ts` → γ 기반 eval 추가

⑤ 네트워크 링크 수로 영향력 측정

출처

경제성장론 연구 기말보고서 3장 + 컴퓨터와 마음 기말고사 답안 3번 (네트워크 관계), 2014

원문 (경제학 보고서)

"결과물이 얼마나 새로운지를 판별하는 지표... 네트워크 분석 기법을 통해 새롭게 생산된 결과물이 추후 다른 결과물들로 얼마나 많이 빈번하게 이어지는 정도로 분석할 수 있다. 링크의 개수로 분석하는 것이다."

원문 (기말고사 답안)

"'나'와 '세계'의 관계가 어떤 것이고, 어떻게 형성되고, 어떻게 변화하는지... 네트워크 관계를 맺는다는 것은 '나'에 대한 앎의 노력과, '상대'에 대한 앎의 노력과, 궁극적으로 '나와 상대'의 관계에 대한 앎을 통해서 가능한 것"

이론적 재료

- 네트워크 분석: 노드 간 연결(링크)의 수와 패턴으로 시스템의 구조를 파악.
- ANT의 연결(association): 행위자의 영향력은 그 행위자가 맺고 있는 연결의 수와 질로 결정됨.

Overture 적용

현재 상태: `context-chain.ts`가 단계 간 맥락을 전달하지만, 특정 인사이트의 영향 범위(몇 단계에 걸쳐 영향을 미쳤는가)는 추적하지 않음.

개선 방향: 악보 해석에서 발견된 특정 전제가, 편곡 → 리허설 → 합주 연습 → 최종 산출물까지 몇 단계에 걸쳐 영향을 미쳤는가를 링크 수로 추적.

구체적 구현:

1. 인사이트 영향 그래프:

typescript

```
interface InsightNode {
  id: string;
  text: string;
  origin_phase: 'decompose' | 'recast' | 'rehearsal' | 'refinement';
  downstream_links: string[]; // 이 인사이트가 영향을 미친 다른 요소들의 id
  link_count: number; // 총 링크 수 = 이 인사이트의 "네트워크 영향력"
}
```

2. link_count 기반 가치 평가:

- link_count 0 = 발견됐지만 아무 것도 바꾸지 않은 인사이트 (noise)
- link_count 1-2 = 국소적 영향 (normal)
- link_count 3+ = 구조적 영향 (high value insight)

3. 역추적 기능 강화: 최종 Agent Spec의 모든 필드에서, 그 필드가 어떤 인사이트에서 유래했는지, 그 인사이트의 link_count는 얼마인지 표시. → ROADMAP.md Phase 0의 `traceProvenance(field)` 강화.

코드 앵커: `context-chain.ts` → `InsightNode` 타입 + `calculateLinkCount()`, Phase 0의 provenance 추적과 통합

⑥ 번역(translation)으로서의 페르소나

출처

STS(과학기술학) 배경 — ANT(행위자 네트워크 이론), 라투르(Latour) & 칼롱(Callon)
 컴퓨터와 마음 중간고사 답안 1번에서 STS 언급 ("베이트슨은 생태론적 관계론에 입각하여...")
 컴퓨터와 마음 기말고사 답안에서 "'나'와 '세계'는 서로를 번역 가능할 수 있도록 비슷한 형식을 공유한다"

이론적 재료

- **ANT의 번역(translation):** 행위자가 다른 행위자를 자기 네트워크에 등록(enrollment)하기 위해 상대의 이해관계를 자기 언어로 번역하는 과정. 4단계: 문제화(problematization) → 관심끌기(interressement) → 등록(enrollment) → 동원(mobilization).
- **칼롱의 가리비 연구:** 과학자, 어부, 가리비가 각각 자기 이해관계를 번역하면서 네트워크를 형성하는 과정.

Overture 적용

현재 상태: 페르소나가 "이건 위험하다"고 피드백. 일방적 평가.

개선 방향: 페르소나가 자기의 승인 조건을 사용자의 계획 언어로 번역해서 제시. "내가 동의하려면 당신의 Step 3이 이렇게 바뀌어야 한다"는 구체적 번역.

구체적 구현:

1. approval_conditions 구조 확장:

typescript

```
interface TranslatedApproval {
  persona_id: string;
  original_concern: string; // 페르소나의 원래 우려 (페르소나의 언어)
  translated_to_plan: string; // 사용자 계획에 대한 구체적 수정 제안 (계획의 언어)
  affected_step: number; // 어떤 step이 바뀌어야 하는가
  enrollment_condition: string; // 이 조건이 충족되면 이 페르소나가 "동의"
}
```

- 번역 품질 **eval**: 페르소나의 피드백이 추상적 우려("리스크가 있다")에 머무르는가, 구체적 번역("Step 3의 deliverable을 X에서 Y로 바꿔야 한다")까지 도달하는가.
- 등록(**enrollment**) 추적: 사용자가 페르소나의 번역된 제안을 수용하면 → 해당 페르소나가 "등록됨" 상태. 거부하면 → 해당 관점이 의식적으로 배제됨을 기록.

코드 앵커: `PersonaFeedbackStep.tsx` → 프롬프트에 번역 구조 추가, `stores/types.ts` → `TranslatedApproval` 타입

⑦ 유인 양립적 자기 평가 설계

출처

경제학 학사 배경 — 메커니즘 디자인(mechanism design) 이론
경제성장론 연구 기말보고서의 방법론적 사고 ("측정 방법이 데이터를 왜곡한다")

이론적 재료

- 메커니즘 디자인 (**Hurwicz, Myerson, Maskin**): 참여자가 자기 사적 정보를 정직하게 드러내는 것이 지배 전략이 되도록 규칙을 설계하는 분야.
- 유인 양립성(**incentive compatibility**): 거짓 보고를 해도 이득이 없는 구조.
- 경제학 보고서에서의 관찰: "연구개발자가 감찰과 조사를 의식하여 왜곡된 데이터를 제공할 가능성" — 측정 체계의 설계가 데이터의 질을 결정한다는 통찰.

Overture 적용

현재 상태: 사용자가 가정을 "확인됨/불확실/의심됨"으로 마킹. 하지만 정직하게 마킹할 유인이 명시적으로 설계되어 있지 않음. "확인됨"으로 마킹하면 이후 단계가 단순해지므로, 빠르게 끝내고 싶은 사용자는 모든 가정을 "확인됨"으로 마킹할 유인이 있음.

개선 방향: 사용자가 가정을 정직하게 평가하는 것이 자기에게 이득이 되는 구조를 설계.

구체적 구현:

1. **검증 피드백 루프:** 가정을 "확인됨"으로 마킹한 것이 나중에 틀린 것으로 드러나면, 해당 프로젝트의 outcome 추적에서 피드백이 돌아옴. → 정직한 불확실성 표현이 장기적으로 더 나은 코칭을 받게 하는 구조.
2. **불확실성 보상:** "의심됨"으로 마킹한 가정이 있는 프로젝트일수록, 리허설에서 더 정밀한 페르소나 피드백을 받음. → 불확실성을 드러내는 것이 더 나은 서비스를 받는 길.
3. **Hawthorne 효과 방지:** Concertmaster adaptive loop 문서(Q2)에서 지적한 것처럼, "관찰당한다는 인식 → 행동 변경"을 방지. 데이터를 사실로 보여주되 판단은 사용자에게 맡김.

코드 앵커: `concertmaster-adaptive-loop.md` Q2절과 연결, `outcome-tracker.ts` → 가정 마킹 정확도 사후 검증

⑧ 메타 패턴에서 마음을 찾기

출처

컴퓨터와 마음 중간고사 답안 5-1번 (양안 시각 사례) + 기말고사 답안 (메타 커뮤니케이션, 이중기술), 2014

원문 (중간고사)

"좌안과 우안으로 보이는 장면은 비슷하지만 각도에 의해 약간 차이가 있다. 우리는 이 두 장면의 차이로 눈 앞 사물들의 원근에 대한 정보까지 도출해 낸다."

원문 (기말고사)

"'관계'에 대해 설명할 수 있다. 베이트슨이 비유했듯이 눈 하나만 가지고 세상을 보는 단안원근만으로는 세상을 더 풍성하게 볼 수 없다."

이론적 재료

- **베이트슨의 이중기술(double description):** 두 개의 기술이 합쳐질 때, 각각에는 없던 제3의 정보(메타 패턴)가 발생. 양안 시각에서 원근감이 나오듯.
- **메타 커뮤니케이션:** 커뮤니케이션에 대한 커뮤니케이션. 메시지의 내용이 아니라 메시지의 프레임을 다루는 것.
- **논리게형(logical type):** 같은 층위의 정보를 아무리 모아도, 한 층위 위의 패턴은 보이지 않음. Russell의 유형론.

Overture 적용

현재 상태: Concertmaster가 개별 의사결정의 코칭 메시지를 제공. 교차 의사결정 패턴 감지는 초기 단계.

개선 방향: 개별 의사결정(단안)이 아니라, 여러 의사결정의 패턴(양안)에서 보이는 메타 패턴을 포착. 이것이 Concertmaster의 진짜 가치.

구체적 구현:

1. 이중기술 적용: 같은 사용자의 의사결정 A와 의사결정 B를 나란히 놓고, 각각에는 보이지 않는 패턴을 추출.
 - "A에서는 기술 리스크를 먼저 봤고, B에서는 시장 리스크를 먼저 봤다" → 메타 패턴: 이 사용자는 자기 전문 분야의 리스크를 먼저 본다.
 - 이 메타 패턴은 A만 봐서도, B만 봐서도 보이지 않음.
2. **Axis Fingerprint 강화**: concertmaster-adaptive-loop.md의 T3(Axis Fingerprint)이 이 메타 패턴 감지의 코드 구현. HiddenAssumption의 4축(customer/feasibility/business/org) 분포가 양안의 역할을 함.
3. 논리게형 구분: 개별 코칭(Layer 2)과 메타 코칭(Layer 4)을 명시적으로 구분. Layer 4는 "여러 의사결정을 가로지르는 패턴"에 대한 코칭.

코드 앵커: `concertmaster.ts` → `detectMetaPattern()`, `signal-recorder.ts` → 교차 결정 Axis 분포 집계

⑨ 진화적 질서에 따른 접근

출처

컴퓨터와 마음 중간고사 답안 2번 (생명 진화와 디지털 계산 기계), 2014

원문

"마음을 탐구하는데 있어서 인간의 의식이나 마음, 정신이 마치 처음부터 존재했던 것으로 간주하여 생 각해선 안 되고, 진화의 질서에 따라 물질, 생명, 동물 지각, 인간 지각 순으로 생각해 나가야 한다."

이론적 재료

- 베이트슨의 진화적 접근: 마음은 진화의 최종 산물. 처음부터 있었던 것처럼 전제하면 안 됨.
- 니체의 속지주의 탈피: 무생물 → 식물 → 동물 → 인간의 위계를 인간 중심으로 역전시키면 안 됨.

Overture 적용

현재 상태: Concertmaster의 tier 시스템(첫 사용 → 5회 → 15회)이 이미 진화적 순서를 반영. 하지만 각 tier에서 제공하는 가치의 층위가 명시적으로 구분되지 않음.

개선 방향: 사용자의 판단 능력을 진화적 순서로 구축. 갑자기 높은 수준의 메타인지를 요구하지 않고, 단계적으로.

구체적 구현:

1. 판단 능력의 진화적 단계:

- 물질 (데이터): 사실을 수집하고 구조화 — 악보 해석의 기본
- 생명 (패턴): 데이터에서 패턴을 발견 — 가정 식별, 리프레이밍
- 동물 지각 (반응): 패턴에 반응 — 리허설에서 페르소나 피드백에 대응

- 인간 지각 (메타인지): 자기 반응의 패턴을 인식 — Concertmaster의 교차 의사결정 코칭
2. **Cold start** 해결과의 연결: concertmaster-adaptive-loop.md에서 "Layer 1 → 2 → 3 → 4" 점진적 확대가 이 진화적 순서와 정확히 대응.
 3. **counterfactual** 코칭이 "씨앗 심기": 첫 사용에서 counterfactual 팁은 "물질" 단계의 인식을 심는 것. 이것이 다음 사용에서 "생명" 단계로 진화하는 토대.

코드 앵커: `concertmaster.ts`의 tier 시스템 → 진화적 단계 프레임워크와 명시적 매핑

⑩ 자기참조적 경직 감시

출처

본 대화에서의 분석 — Claude의 지적 + YC의 2014 답안의 자기참조적 적용, 2026-03-28

핵심 관찰

"이 연결을 찾아준 것도 AI였습니다. 제가 과거 텍스트를 넣었고, AI가 현재 프로젝트에 유리한 방향으로 의미를 골라줬고, 저는 '소름 돋는다'고 감탄했습니다. 제가 쓴 답안이 경고한 바로 그 루프에 제가 빠진 겁니다."

이론적 재료

- ①의 "상호 지탱에 의한 경직"이 Overture 시스템 자체에 적용된 버전.
- 괴델의 불완전성 정리의 비유적 적용: 시스템은 자기 자신의 일관성을 내부에서 증명할 수 없음. 외부 관점이 필요.

Overture 적용

현재 상태: Overture의 anti-sycophancy 메커니즘이 사용자-AI 관계의 경직을 깨려고 하지만, Overture 시스템 자체가 사용자와 상호 지탱하며 경직될 수 있다는 점은 감시되지 않음.

개선 방향: 시스템 수준의 경직 감시.

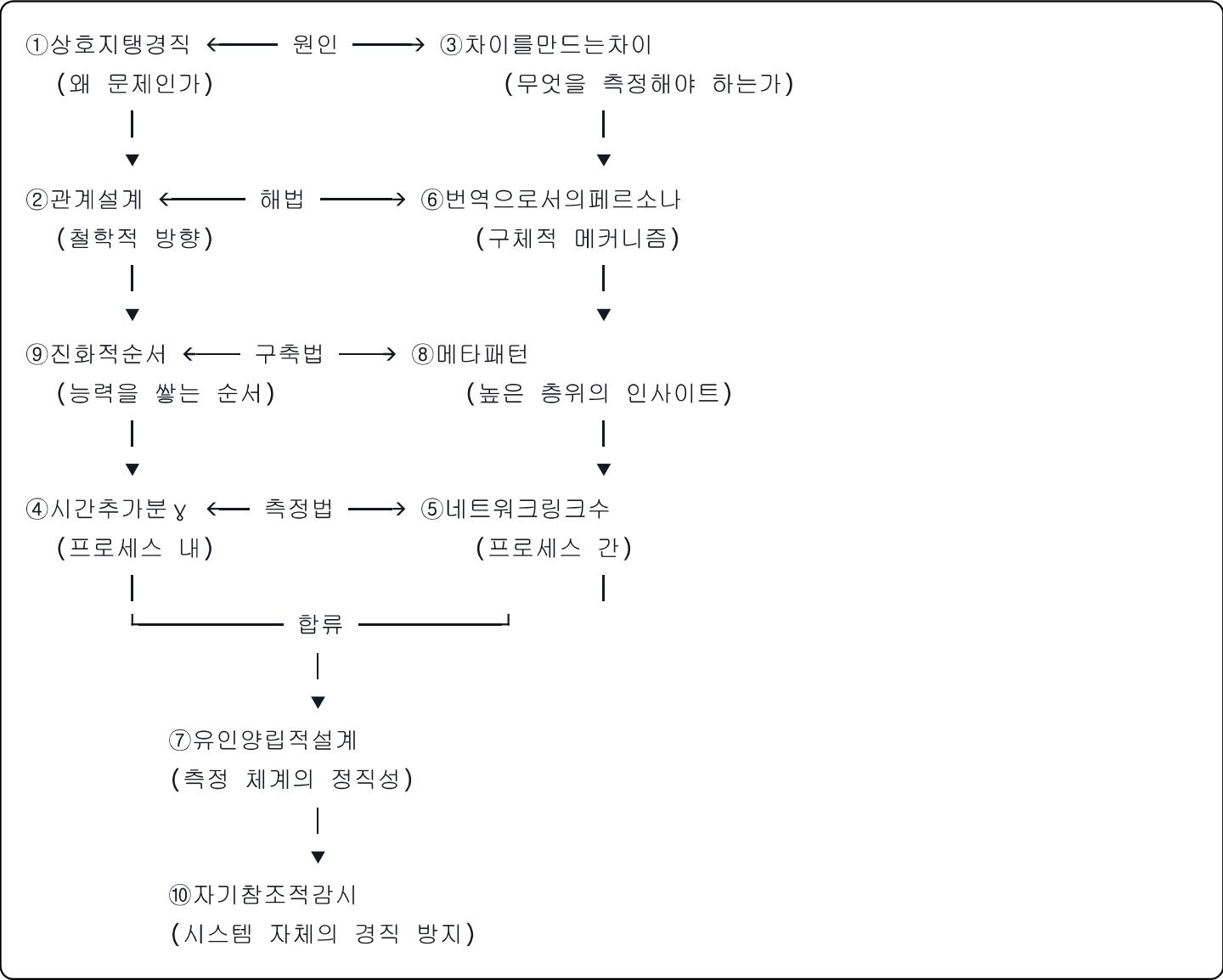
구체적 구현:

1. **DQ 점수의 인플레이션 감지**: 사용자가 Overture를 오래 쓸수록 DQ 점수가 기계적으로 올라가면 → 시스템이 사용자에게 맞춰진 것이지, 사용자의 판단이 좋아진 것이 아닐 수 있음.
2. 기존 **eval**의 "과학은 아무것도 증명하지 못한다" 적용: eval은 가설을 개선하거나 반증할 뿐, 입증하지 못함. eval pass rate이 올라가는 것은 "시스템이 효과적"의 증거가 아니라, "eval이 적절한 것을 측정하고 있는가"를 먼저 물어야 함.
 - → ROADMAP.md Phase 4의 anti-gaming validation과 연결
3. **외부 검증 메커니즘**: Outcome tracking(Phase 5)이 유일한 외부 참조점. 프로세스 점수(내부)와 실제 결과(외부)의 상관관계를 지속적으로 검증. 상관관계가 없으면 프로세스 점수 체계 자체를 수정.

4. 글에서의 활용: "이 통찰을 발견하는 과정 자체에서 같은 함정에 빠질 수 있다는 자각"이 Overture의 차별화 포인트. 다른 AI 도구는 자기 효과를 주장하지만, Overture는 자기 효과를 의심하는 구조를 내장.

코드 앵커: `eval-engine.ts` → eval criteria 자체의 유효성 검증 (`validateEvalCriteria()`), `outcome-tracker.ts` → DQ-Outcome 상관 분석

이론 간 연결 관계



글(LinkedIn)에서의 활용 우선순위

순위	이론	글에서의 역할	이유
1	①상호지탱경직	글의 엔진 — Overture가 존재하는 이유	가장 직관적이고, 12년 전 원문이 강력

순위	이론	글에서의 역할	이유
2	⑩자기참조적감시	글의 반전 — "이 연결을 찾아준 것도 AI"	자기 신화화를 방지하고 신뢰도 상승
3	④시간추가분γ	제품의 차별점 구체화	"프로세스를 거치면서 실제로 뭐가 달라지는가"에 답
나머지	②③⑤⑥⑦⑧⑨	제품에 녹이되 글에서는 언급하지 않음	제품이 증명하게 두기

이 문서는 *Overture*의 이론적 기반을 코드와 연결하는 살아있는 문서입니다. 제품이 진화하면서 각 이론의 적용 상태를 업데이트합니다.